

Relazione — Versione 1: Agent Base

Obiettivo della Versione 1

La Versione 1 rappresenta il primo passo nella costruzione dell'agente AI.

L'obiettivo principale è creare un agente semplice ma funzionante, capace di ricevere una richiesta dell'utente, capire quale azione eseguire, estrarre gli argomenti necessari, eseguire il tool corretto e salvare il risultato in memoria.

Questa versione non punta ancora a una struttura professionale completa. Serve soprattutto per capire il flusso base di un agente.

Cosa fa l'agente nella Versione 1

Il flusso principale è:

```
User request
↓
Router
↓
Parser
↓
Agent
↓
Memory
```

L'utente invia una richiesta, per esempio:

```
What is 2 + 3?
```

L'agente deve:

1. capire che l'utente vuole fare una somma;
2. scegliere il tool `addition`;
3. estrarre i numeri `2` e `3`;
4. eseguire direttamente la funzione;
5. salvare lo stato dell'interazione nella memoria.

Il risultato finale è uno stato simile a questo:

```
{
  "request": "What is 2 + 3?",
  "action": "addition",
```

```
"arguments": {"a": 2, "b": 3},  
"result": 5  
}
```

Componenti presenti

Tools

I tools sono semplici funzioni Python.

Esempi:

```
addition(a, b)  
subtraction(a, b)  
multiplication(a, b)  
division(a, b)  
power(a, b)  
greeting(name)
```

Ogni tool rappresenta una capacità dell'agente.

In questa versione i tool sono ancora semplici, ma servono a capire il concetto fondamentale:

un agente non fa tutto da solo, ma usa strumenti.

Parser

Il Parser ha il compito di estrarre informazioni dalla richiesta dell'utente.

Per esempio:

```
What is 2 + 3?
```

diventa:

```
{"a": 2, "b": 3}
```

Oppure:

```
Hello, my name is luca.
```

diventa:

```
{"name": "luca"}
```

In questa versione il parser è ancora semplice e usa regole rigide.

Router

Il Router sceglie quale tool usare.

Per esempio:

```
What is 2 + 3?
```

porta a:

```
"addition"
```

Mentre:

```
Hello, my name is luca.
```

porta a:

```
"greeting"
```

Il Router usa parole chiave e sinonimi per capire l'intento dell'utente.

Agent

Nella Versione 1 l'Agent coordina tutto, ma esegue anche direttamente il tool.

Questo significa che l'agente ha ancora troppe responsabilità:

- sceglie il tool;
- chiama il parser;
- valida gli argomenti;
- esegue la funzione;
- salva il risultato in memoria.

Per una prima versione va bene, ma nelle versioni successive questa responsabilità verrà separata.

Memory

La memoria è una semplice lista:

```
memory = []
```

Ogni interazione viene salvata come dizionario.

Questo permette di vedere cosa è successo nelle richieste precedenti.

Cosa abbiamo imparato nella Versione 1

La Versione 1 insegna il flusso base di un agente:

```
capire → estrarre → eseguire → salvare
```

È il primo scheletro funzionante.

La cosa importante non è ancora la perfezione del codice, ma capire come i pezzi principali collaborano.

Limiti della Versione 1

La Versione 1 ha alcuni limiti naturali:

- l'agente esegue direttamente i tool;
- il parser è rigido;
- la memoria è solo una lista;
- non esiste ancora un planner;
- non c'è ancora una LLM;
- la struttura è ancora tutta dentro il notebook.

Questi limiti sono normali.

Servono proprio a preparare le versioni successive.

Conclusione

La Versione 1 è la base del progetto.

Abbiamo costruito un agente semplice, funzionante e comprensibile.

Il suo scopo è imparare il comportamento minimo di un agente:

ricevere una richiesta, scegliere un'azione, preparare gli argomenti, eseguire e salvare il risultato.

Da qui possiamo iniziare a separare meglio le responsabilità.

Il passo successivo sarà la Versione 2, dove introdurremo l'Executor.